

Practical Work No. 1: Directed Graph Documentation C++

Student: Giurgiu Corina Maria

Date: March 24, 2026

1. Design & Internal Representation

- **Adjacency List (outbound):** A `std::vector` of `std::unordered_map<int, Edge*>`. This allows $O(1)$ average-case access to a vertex's neighbors.
- **Inbound Map (inbound):** A corresponding `std::vector` of `std::unordered_map` to track predecessors, ensuring "who points to a vertex" is also an $O(1)$ operation.
- **Edge Objects (cost):** Edges are stored as pointers to `Edge` structs. This allows the `inbound` and `outbound` maps to point to the **same object in memory**, making cost modifications instant for both directions.

2. Complexity Analysis

Operation	Complexity	Reason
add_edge / remove_edge	$O(1)$ average	Hash map insertion/deletion.
is_edge	$O(1)$ average	Direct hash map lookup.
get_in / out_degree	$O(1)$	Direct access to the <code>size()</code> of the map.
remove_vertex	$O(\deg(v))$	Iterates only through the incident edges of the vertex.
add_vertex	$O(1)$ amortized	Vector <code>push_back</code> operation.

3. Memory Management (The Rule of Three)

Because the graph uses raw pointers (`Edge*`) to ensure the `inbound` and `outbound` maps stay synchronized, the following were implemented to prevent memory leaks and segmentation faults:

- **Destructor (~Graph):** Properly deletes every edge pointer to free memory when the graph object is destroyed.
- **Copy Constructor:** Performs a **Deep Copy**. It creates entirely new `Edge` objects in memory so that modifications to a copy do not affect the original graph.

- **Assignment Operator:** Handles self-assignment and clears existing heap memory before copying new data from another graph.

4. Requirement Verification

- **Random Graphs:** Two graphs were generated and saved:
 1. `random_graph1.txt`: 6 vertices, 20 edges.
 2. `random_graph2.txt`: 7 vertices, 40 edges.
- **Large Files:** The C++ implementation was tested against `graph100k.txt`, showing significantly faster performance than Python due to the lower overhead of STL containers.
- **Preconditions:** All methods (like `get_in_degree` or `parse_out`) include bounds checking to ensure vertex IDs are within the valid range $[0, n-1]$.

5. Method Documentation

- **`Graph(int n)`:** Creates a graph with n vertices and 0 edges.
- **`add_vertex()`:** Adds a new vertex with the next available ID.
- **`remove_vertex(int v)`:** Removes a vertex. It takes a "snapshot" of edges first to avoid iterator invalidation during removal.
- **`is_edge(u, v)`:** Returns true if there is a directed edge from u to v .
- **`parse_out(v)` / `parse_in(v)`:** Returns a constant reference to the maps, allowing iteration over neighbors without exposing the internal structure to modification.
- **`create_random_graph(n, m)`:** Generates a random graph, with a safety cap to ensure $m \leq n^2$.

6. Implementation

```
// Internal structures used for the Graph ADT
struct Edge {
    int source; // starting vertex
    int target; // ending vertex
    int cost;
};

typedef Edge* Edge_id; // Abstract identifier for edges

// Data members of the Graph class
class Graph {
private:
    int nr_vertices;
    int nr_edges;
    // Map of outbound edges: source -> (target -> Edge_id)
```

```
std::vector<std::unordered_map<int, Edge_id>> outbound;  
// Map of inbound edges: target -> (source -> Edge_id)  
std::vector<std::unordered_map<int, Edge_id>> inbound;  
...  
};
```